GCash FinTech Engineer Specialist Course

Major Course Output 1 (MCO1)

Monolith Project Specifications

Course Type	Elective Computing Course	**Output Weight**	20% of the overall course grade
Mode	Individual	**Learning Outcomes**	LO1, LO2, LO3
Coverage	Weeks 1 to 6 topics	**Deadline**	**On or before October 28, 2026**

1. Overview

MCO1 requires each student to independently design, build, and demonstrate a monolithic enterprise backend application using Java Spring Boot. The project serves as evidence that the student can build production-ready RESTful APIs, implement reliable data persistence, and secure an application using Spring Security and JSON Web Tokens (JWT). It applies all topics covered from Week 1 to Week 6 of the course.

2. Learning Outcomes Assessed

- **LO1** – Build and configure production-ready RESTful APIs using Spring Boot.
- **LO2** – Implement robust data persistence using Spring Data JPA and relational databases.
- **LO3** – Secure enterprise applications using Spring Security and JWT.

3. Relationship to MCO2

MCO1 is the foundation of the course capstone. The flow of the two major course outputs is as follows:

Stage	Description
MCO1 (Individual)	Every student individually develops and submits a monolith application. MCO1 is a submission-only output.
Group Formation	After MCO1, students group themselves into teams of five (5) members. Each group selects one (1) member's MCO1 project to be upgraded to a microservices architecture.
MCO2 Development	Each group upgrades the chosen monolith into microservices, following the rule of one (1) microservice per group member, for a total of five (5) microservices per group.
MCO2 (Group)	The final microservices application is presented through a demonstration and defense (D&D) with GCash panels.

Because any MCO1 project may be selected as the basis for a group's MCO2, students are encouraged to produce a clean, well-structured, and well-documented monolith that can be decomposed into bounded contexts later in the term.

4. Technical Scope

The monolith must apply the topics from Weeks 1 to 6. At minimum, the project is expected to demonstrate the following:

Week	Topic	Expected Application in the Project
1–2	Spring Boot, IoC, Dependency Injection; RESTful Web Services and Routing	A Spring Boot project bootstrapped via Spring Initializr; constructor-based dependency injection; REST controllers using proper HTTP methods and status codes; use of the DTO pattern.
3	Data Persistence with Spring Data JPA and Hibernate	JPA entities and repositories connected to a MySQL database configured through application.properties; full CRUD operations on the domain entities.
4	Exception Handling and Bean Validation	Validation annotations (e.g., @NotBlank, @Email) on request DTOs; centralized error handling using @ControllerAdvice and @ExceptionHandler; consistent custom error responses.
5	Spring Security and JWT Authentication	A login endpoint that issues JWTs; secured CRUD endpoints; proper use of UserDetails, UserDetailsService, PasswordEncoder, and SecurityFilterChain; role- or method-level authorization where appropriate.
6	Testing in Spring Boot (JUnit and Mockito)	Unit tests for services using Mockito and integration tests for controllers (e.g., MockMvc, slice tests).

5. Software Tools

- Java Development Kit (JDK) 17 or 21 (LTS) and Spring Initializr
- IntelliJ IDEA (recommended), Eclipse IDE for Enterprise Java, or Visual Studio Code with the Spring Boot Extension Pack
- Apache Maven for dependency management and packaging
- MySQL Community Server, with MySQL Workbench or DBeaver for verifying persisted data
- Postman or Insomnia for testing endpoints and JWT authentication headers
- Draw.io, Lucidchart, or PlantUML for the UML class diagram
- Git and GitHub for version control and submission

6. Deliverables

Each student must submit all of the following. All deliverables must be compiled into a single GitHub repository.

#	Deliverable	Requirements
1	**Monolith Project (Source Code)**	A complete, runnable Spring Boot monolith applying the Week 1 to Week 6 topics, with all CRUD operations and authentication functional and error-free. Code must follow the course coding standards and include Javadoc for every method (with pre-conditions, post-conditions, parameters, and return values) and in-line comments on long blocks of code.

#	Deliverable	Requirements
2	**UML Class Diagram**	A visualization of the system structure showing all classes with their attributes, behaviors (methods), and relationships. The implementation must be consistent with the submitted diagram.
3	**Test Cases and Test Scripts**	Every method in every class must be tested with at least three (3) documented, unique test cases.
4	**Recorded Demonstration**	A recorded, hands-on demonstration of the working project. The student must show full mastery of the application's design and implementation.

7. Submission Guidelines

- **Deadline:** on or before **October 28, 2026**.
- **Repository:** compile all deliverables (source code, UML class diagram, test scripts, and the recorded demonstration or a link to it) in one GitHub repository. Ensure the repository is accessible to the instructor.
- **Submission:** enter the GitHub repository link in the **MCO1 Submission** on Canvas.
- Include a README in the repository that explains how to configure the database and run the application.

8. Grading Summary

MCO1 is scored out of 100 points using the rubric in Appendix A. This score is then weighted to account for **20% of the overall course grade** (for example, a rubric score of 85/100 contributes 17% to the final grade).

Criterion	Points	Focus
Program Correctness and Completeness	45	Everything runs error-free; all CRUD and authentication functional.
Designing Classes/Objects	20	A logical UML class diagram.
Consistency of Design and Code	15	The implementation matches the submitted design.
Readability and Documentation	10	Course coding standards, Javadoc with pre- and post-conditions, in-line comments on long code.
Test Case Design and Documentation	10	Every method in every class, three or more documented unique cases.
Delivery and Presentation	10	Full mastery shown in the demonstration.
Total	**100**	

9. Prerequisite Conditions

Meeting the following conditions is a prerequisite for receiving any score. If **any** of the following applies, the **entire MCO1 output receives a score of zero (0)**:

- Late submission of deliverables.
- Part or all of the deliverable is plagiarized or is not the product of the student's own work.
- The student did not appear during the demonstration.

Appendix A – MCO1: Monolith Project Rubric

Total: 100 points

Criteria	Exemplary	Satisfactory	Developing	Beginning	None
[Prerequisite] **100%**	Late submission of deliverables. **OR** Part or all of deliverable is plagiarized or not a product of the student's output. **OR** No significant contribution to the group output. **OR** Did not appear during the demo. **Score: 0**				
Program Correctness and Completeness	All CRUD and Auth functional; error-free. **40 – 45**	Missing minor features. **30 – 40**	Missing significant features. **15 – 30**	Major errors; incomplete. **0 – 15**	Program does not run **OR** Program does not produce any output. **0**
Designing Classes/Objects	Design decisions comply completely with the specs, and all classes, attributes, behaviors, and relationships identified and shown in the UML class diagram are logical. **20**	Design decisions comply completely with the specs as shown in the UML class diagram, but include unnecessary or redundant classes **OR** there is incomplete information on the attributes, behaviors, or relationships. **15**	The design provides for most but not all of the original specs as shown in the UML class diagram. Most likely includes unnecessary or redundant classes **AND** there is incomplete information on the attributes, behaviors, or relationships. **10**	The UML class diagram does not include most information based on the specs. **5**	No submitted UML class diagram. **OR** Design of classes is not in compliance with a logical object-based design. **0**
Consistency of Design and Code	Program implementation corresponds correctly with the presented object-based design. **15**	There are minor inconsistencies in design or implementation of attributes or methods, but all necessary features are still provided. **12**	There are minor inconsistencies in design or implementation of attributes or methods, which lead to missing or unexpected features. **8**	There are significant inconsistencies between design and implementation at the class level, but most features are still apparent. **5**	No submitted class diagram to compare with. **0**

Criteria	Exemplary	Satisfactory	Developing	Beginning	None
Readability and Documentation	Coding standards prescribed in the course are followed. **AND** In-line comments are included for long code, apart from proper documentation of methods (inclusive of pre-conditions, post-conditions, method parameters, and return data) via Javadoc. **10**	Coding standards prescribed in the course are followed. **AND** No in-line comments are included for long code, but there is proper documentation of methods (inclusive of pre-conditions, post-conditions, method parameters, and return data) via Javadoc. **8**	Coding standards prescribed in the course are followed. **AND** No in-line comments are included for long code. There is method documentation via Javadoc, but some information is missing. **5**	Coding standards prescribed in the course are followed. **AND** No in-line comments are included for long code. **AND** Method documentation is not via Javadoc annotation. **3**	No internal documentation. **0**
Test Case Design and Documentation	All methods in all classes are tested with 3+ unique cases. **10**	All methods in all classes are tested, but not all have at least 3 documented unique test cases. **8**	Most methods in all classes are tested with at least 3 documented unique test cases. **5**	Many methods do not have at least 3 documented unique test cases. **3**	No test script submitted. **0**
Delivery and Presentation	Full mastery of the Monolith Application. **10**	Exhibits familiarity with the project throughout the demo. **8**	Unequal contribution. Exhibits familiarity with the project throughout the demo. **5**	Exhibits some knowledge of the project throughout the demo. **3**	Did not appear in the demo. **OR** Did not exhibit ample knowledge about the design **AND** implementation of the project. **0**

Note: The prerequisite criterion applies to the whole output. If any of its conditions is met, the entire MCO1 score is zero regardless of performance on the other criteria.